Learning Roadmap

From first principles to expert practice

Built from the manuscript sources · 0 chapter(s) in this edition

Appendix E — Learning Roadmap and Proficiency Levels

E.1 How to use this roadmap

E.2 The five proficiency levels

E.3 Self-assessment

E.4 Track 1 — Full 12-month program (beginner to Level 4)

E.5 Track 2 — 16-week interview track (Level 2–3 engineer → senior interviews)

E.6 Track 3 — 24-week ML/AI infrastructure track (software engineer → ML/AI systems)

E.7 Monthly milestone review template

2. Concept Dependency Map

2.1 Part-level dependency graph

2.2 Concept-level dependency graph (critical path)

2.3 Chapter prerequisite table

2.4 Reading paths

Appendix E — Learning Roadmap and Proficiency Levels

> **Deliverable 3.** This appendix turns the 93 chapters into a schedule. It defines five proficiency levels, what you must know and be able to do at each, and week-by-week milestones for three tracks: the full 12-month program, a 16-week interview track, and a 24-week ML/AI infrastructure track for working engineers.

E.1 How to use this roadmap

1. Find your starting level with the self-assessment in §E.3. Do not skip a level because a topic "sounds familiar" — the readiness criteria test whether you can *explain and apply* a concept, not whether you recognize its name.
2. Pick a track (§E.4–E.6). Each week lists chapters, a hands-on task and a checkpoint. Budget the stated hours; if a week overruns, extend it rather than skipping the exercise — implementation is where understanding consolidates (Ch 3, §3.7).
3. At the end of every month, run the month's **milestone review**: answer the knowledge checkpoints of each chapter from memory, redraw the month's key architecture diagrams without notes, and solve one design challenge aloud in 45 minutes.
4. Record progress in a design journal (template in Ch 3, §3.10).

E.2 The five proficiency levels

Figure E.1 — Proficiency ladder.

WARNING / TRADE-OFF

Diagram failed to render. Source:

```
flowchart LR
  L1["Level 1<br/>Absolute beginner"] --> L2["Level 2<br/>Intermediate engineer"]
  L2 --> L3["Level 3<br/>Advanced engineer"]
  L3 --> L4["Level 4<br/>Senior / Staff"]
  L4 --> L5["Level 5<br/>Expert / Principal"]
```

Reading the diagram. The levels are cumulative: each assumes every competency of the previous one. The arrows are not time-boxed — a strong backend engineer may al-

ready sit at Level 3 for traditional systems but at Level 1 for ML infrastructure. Assess each track (A: traditional, B: ML/AI) separately.

Level 1 — Absolute beginner

Dimension	Definition
Required knowledge	Ch 1–8, 11, 35. What a client, server, API, database, process, thread, IP address, DNS lookup and HTTP request are; functional vs non-functional requirements; powers of ten and basic percentiles.
Expected competencies	Trace one HTTP request from browser to database and back, naming every hop. Write a CRUD REST API with a relational database. Model a simple domain in tables with primary/foreign keys. Do a back-of-the-envelope estimate of requests per second from daily active users.
Practical projects	P-01 REST API service; P-02 URL shortener (single node).
Architecture exercises	Draw a single-server web app; then add a load balancer and a second app server and explain what breaks (sessions).
Interview readiness	Not yet ready for system design rounds. Can answer “what happens when you type a URL?” thoroughly.
Explain independently	TCP vs UDP; what an index is and why it speeds reads but slows writes; what latency vs throughput mean; why HTTPS needs certificates.

Level 2 — Intermediate engineer

Dimension	Definition
Required knowledge	Ch 9–10, 12, 14, 17–22, 24, 30, 33, 36–37, 57. Replication, sharding, consistent hashing, caching patterns, CDNs, load-balancing algorithms, rate limiting, queues and pub/sub, monolith vs microservices, ML basics.
Expected competencies	Turn a vague prompt into numbered FRs/NFRs with targets. Estimate storage, bandwidth and RPS with peak factors. Choose SQL vs NoSQL with justification. Add a cache and explain invalidation and stampede protection. Design a rate limiter and pick an algorithm.
Practical projects	P-03 Redis cache; P-04 rate limiter; P-05 load-balancer simulation; P-06 message queue.
Architecture exercises	URL shortener, Pastebin, rate limiter and notification system (Ch 52) at 10^7 DAU.
Interview readiness	Can complete an entry-level (new-grad / mid-level) system design round: correct high-level architecture, reasonable estimates, identifies the main bottleneck.
Explain independently	Cache-aside vs write-through; why sharding needs a good key; at-least-once delivery and why consumers must be idempotent; Little’s Law applied to a thread pool.

Level 3 — Advanced engineer

Dimension	Definition
Required knowledge	Ch 13, 15–16, 23, 25–27, 29, 31–32, 34, 38–43, 58–62, 66. Storage engines, consistency models, Raft, quorums, Kafka internals, outbox/saga/CQRS, observability, Kubernetes, CI/CD, ML lifecycle, feature stores, model serving.
Expected competencies	Design a complete distributed application with explicit consistency choices, failure handling and observability. Explain a database's write path down to WAL and fsync. Design an ML system from problem formulation to online serving with a feature store.
Practical projects	P-07 distributed KV store; P-08 replicated DB simulation; P-09 notification service; P-10 real-time chat; P-11 distributed job scheduler.
Architecture exercises	All of Ch 52–53; ML case studies 81.5 (product recommendation) and 82.3 (spam detection).
Interview readiness	Passes a senior-level (L5/E5 equivalent) traditional round and a mid-level ML system design round: deep-dives on request, quantifies trade-offs, discusses failure modes unprompted.
Explain independently	Raft leader election and log matching; why `R + W > N` gives read-your-writes only under certain conditions; Kafka ISR and `acks=all`; point-in-time feature joins and training–serving skew.

Level 4 — Senior / Staff engineer

Dimension	Definition
Required knowledge	Ch 28, 44, 46–48, 50–51, 63–65, 67–80. Distributed transactions, multi-region design, cluster scheduling, multi-tenancy, security architecture, cost; GPU performance, distributed training, model optimization, recommendation architecture, MLOps, LLM inference, RAG, agents, AI platforms.
Expected competencies	Make and defend architecture decisions spanning teams: multi-region active–active, tenant isolation, SLOs and error budgets, cost models. Size a GPU fleet for training and for LLM serving from first principles. Design a RAG or agent platform with evaluation, guardrails and observability.
Practical projects	P-12 event-driven microservices; P-13 real-time analytics pipeline; P-14 recommendation engine; P-15 distributed ML inference; P-18 production RAG; P-19 scalable LLM inference.
Architecture exercises	All of Ch 54–56 and 81–86. Write a design document with ADRs for one case study and have it reviewed.
Interview readiness	Passes staff-level rounds: drives the conversation, scopes ambiguity, prioritizes the hardest sub-problem, connects technical choices to business, cost and organizational constraints.
Explain independently	Spanner commit-wait; why cells limit blast radius; ZeRO stage 3 memory math; continuous batching and PagedAttention; when RAG beats fine-tuning; how to evaluate an agent.

Level 5 — Expert / Principal architect

Dimension	Definition
Required knowledge	Ch 45, 49, 80 plus the primary papers cited across Parts V, IX, XIII and XVI. Theory (FLP, CAP proofs, BFT bounds, consistency hierarchies, scaling laws) and practice at 10^4–10^5 machine scale.
Expected competencies	Set multi-year technical direction; evaluate novel systems from papers; reason about organization-wide failure domains, reliability economics and sustainability; design platforms other teams build on.
Practical projects	P-16 distributed training platform; P-17 multi-tenant feature store; P-20 multi-agent orchestration platform.
Architecture exercises	Critique a published architecture (e.g., a Dynamo, Spanner or PagedAttention paper) and propose a design for a changed constraint. Lead a postmortem-driven redesign.
Interview readiness	Principal-level: articulates long-term trade-offs, migration strategy, organizational design (Conway's law), and the economics of the platform.
Explain independently	Why consensus is impossible in a fully asynchronous system with one crash failure and how real systems sidestep it; goodput at 16k-GPU scale; the cost/latency frontier of an LLM product.

E.3 Self-assessment

Score yourself 0–2 on each statement (0 = cannot, 1 = with notes, 2 = from memory, aloud, in under five minutes). Start at the first level where your total is below 14 of 20.

Table E.1 — Placement self-assessment.

Level	Ten statements to score
1	Explain DNS resolution · TCP handshake · HTTP request/response · process vs thread · what an index is · primary/foreign keys · REST verbs and status codes · latency vs throughput · p99 · RPS from DAU
2	Cache-aside and invalidation · consistent hashing · token bucket · L4 vs L7 load balancing · replication lag · sharding keys · at-least-once delivery · CDN pull vs push · SQL vs NoSQL choice · precision vs recall
3	Raft election · quorum math · Kafka partitions/ISR · outbox pattern · LSM vs B-tree · Kubernetes control plane · SLO/error budget · feature store PIT join · dynamic batching · canary analysis
4	2PC failure modes · multi-region active–active · tenant isolation options · cost model of a service · data/tensor/pipeline parallelism · KV-cache memory · RAG evaluation · agent guardrails · model routing · DRF scheduling
5	FLP intuition · linearizability vs serializability · PBFT bound · TrueTime · ZeRO-3 communication · MFU at scale · scaling laws · disaggregated serving · failure rates at 10k GPUs · multi-year platform migration

E.4 Track 1 — Full 12-month program (beginner to Level 4)

Budget: 10–12 hours per week · 52 weeks · Levels 1 → 4. Each week: read (≈ 60%), implement or exercise (≈ 30%), recall and explain (≈ 10%).

Figure E.2 — Twelve-month program at a glance.

WARNING / TRADE-OFF

Diagram failed to render. Source:

```
gantt
  title Twelve-month program
  dateFormat  YYYY-MM-DD
  axisFormat  M%m
  section Level 1
  Thinking, maths, CS foundations      :a1, 2027-01-04, 8w
  Web, data basics, OOD                :a2, after a1, 5w
  section Level 2
  Data systems, scale, caching         :b1, after a2, 7w
  Distributed basics, messaging        :b2, after b1, 5w
  section Level 3
  Distributed depth, Kafka, EDA        :c1, after b2, 5w
  Production infra, case studies I-II  :c2, after c1, 6w
  section Level 3-4 ML
  ML foundations, data, GPUs, serving  :d1, after c2, 8w
  section Level 4
  LLMs, RAG, agents, AI infra          :e1, after d1, 5w
  Expert topics, case studies, interviews :e2, after e1, 3w
```

Reading the diagram. Bars are sequential phases; the dates are an example start (any Monday works). Levels overlap at the boundaries because review weeks consolidate the previous level while starting the next. The ML phase begins only after production infrastructure because serving, MLOps and AI-infrastructure chapters assume Kubernetes, observability and deployment knowledge.

Table E.2 — Weekly milestones, 12-month program.

Wk	Chapters	Hands-on task	Checkpoint
1	1, 3	Set up design journal and flashcard deck	Explain FR vs NFR with three examples each
2	2	Write requirements + context diagram for a food-ordering app	Present it in 10 minutes
3	4 (§4.3–4.9)	Estimation drills: 20 powers-of-ten problems	Score ≥ 18/20
4	4 (§4.10–4.18)	Simulate an M/M/1 queue in Python; compare with formula	Explain why latency explodes near 100% utilization
5	5	Cache-stride benchmark	Draw the memory hierarchy with latencies from memory
6	6	Race condition + mutex fix; event-loop echo server	Explain epoll vs thread-per-connection
7	7	`dig`, `curl -w`, Wireshark on a TLS handshake	Narrate “type a URL” end to end
8	Review	Redo checkpoints Ch 1–7	**Month 2 milestone review**
9	8	P-01 REST API with pagination and JWT	Defend cursor vs offset pagination
10	11	Schema design + `EXPLAIN` on indexed vs unindexed queries	Explain B+ tree lookups in I/Os
11	35, 36	Refactor a small codebase with SOLID	Class diagram from memory
12	9, 10	SSE + WebSocket demo; password hashing	Choose a real-time technique for 3 scenarios
13	Review	P-02 URL shortener (single node)	**Level 1 exit**: self-assessment ≥ 14
14	12	Model one domain in a document DB and a wide-column DB	SQL vs NoSQL decision for 5 prompts
15	13	Toy LSM tree (memtable + SSTables + compaction)	Draw write path with WAL
16	14	Consistent-hashing ring with virtual nodes	Keys moved when adding a node
17	15, 16	Inverted index + BM25 in Python	Explain erasure coding overhead
18	17	Bloom filter, HyperLogLog, Snowflake IDs	False-positive-rate calculation

Wk	Chapters	Hands-on task	Checkpoint
19	18, 19	P-05 load-balancer simulation	Power-of-two-choices intuition
20	20, 21	P-03 Redis cache-aside with stampede protection	Five caching patterns from memory
21	22	P-04 distributed rate limiter	Token bucket parameters for a given SLA
22	23	Load test with `wrk` / `k6`; find the bottleneck	Explain coordinated omission
23	24	Lamport and vector clocks implementation	Two partial-failure stories
24	30	P-06 message queue with acks and DLQ	Delivery semantics table from memory
25	33, 37	Implement 6 design patterns	Monolith vs microservices debate
26	Review	Ch 52.1–52.4 case studies	**Level 2 exit** + **Month 6 milestone**
27	25	Jepsen-style anomaly hunt on a toy store	Consistency hierarchy diagram
28	26	Raft (election + log replication) — MIT 6.5840 style	Explain fencing tokens
29	27, 28	P-07 distributed KV store with quorums	2PC failure walkthrough
30	29, 31	Kafka locally: partitions, consumer groups, lag	Partition count sizing
31	32, 34	Outbox + saga demo	Choreography vs orchestration
32	38	Four LLD problems (parking lot, elevator, LRU, vending)	45-min LLD mock
33	39, 40	SLOs, burn-rate alerts, OpenTelemetry tracing	Error-budget policy
34	41, 42	Deploy P-01 to Kubernetes with HPA	Kubernetes control-plane walk-through
35	43	CI/CD with canary; Terraform a VPC	Rollback plan
36	52, 53	P-09 notification service; P-10 chat	Two 45-min mocks
37	54, 55	YouTube, Uber, payment system cases	Two 45-min mocks
38	Review	56 + P-11 job scheduler	**Level 3 exit (Track A)**

Wk	Chapters	Hands-on task	Checkpoint
39	57	Train/evaluate a classifier; compute all metrics	Precision/recall trade-off for fraud
40	58, 59	Small MLP in PyTorch; memory math	ML lifecycle diagram
41	60, 61	Batch + streaming feature pipeline	Lambda vs Kappa
42	62, 63	Feast feature store with PIT join	Training–serving skew causes
43	64	Roofline analysis of a matmul	Compute- vs memory-bound
44	65	DDP on 2 GPUs (or simulated); FLOPs estimate	6ND estimate for a model
45	66, 67	Serve a model with dynamic batching; quantize it	Batch size vs latency curve
46	68–71	P-14 two-tower recommender; drift monitor	Multi-stage funnel sizing
47	72	Implement attention + KV cache in NumPy	KV-cache size for a 70B model
48	73	Run vLLM; measure TTFT/TPOT vs concurrency	Continuous batching explanation
49	74, 75	P-18 production RAG with hybrid search + reranking	Retrieval metrics
50	76–78	Agent with tool calling, guardrails and eval set	Prompt-injection defenses
51	79, 80, 81–86 (2 cases)	Model gateway with token-based rate limits	Two ML/AI mocks
52	87–92	Full mock loop: 2 traditional + 2 ML/AI	**Level 4 readiness review**

Level 5 content (Ch 45, 49, 80 deep dives, primary-paper reading) is a continuing program after week 52: one paper and one design critique per week.

E.5 Track 2 — 16-week interview track (Level 2–3 engineer → senior interviews)

Budget: 12–15 hours/week. Assumes Level 2 fundamentals.

Table E.3 — Interview track.

Wk	Focus	Chapters	Practice
1	Framework and estimation	2, 4, 87, 88, 89	30 estimation drills
2	Data systems refresh	11, 12, 14	Q-T-01 … Q-T-04
3	Caching, CDN, LB	19, 20, 21	Q-T-05 … Q-T-08
4	Traffic management	22, 17	Rate limiter + URL shortener mocks
5	Distributed core	24, 25, 26	KV-store mock
6	Replication and transactions	27, 28	Payment-system mock
7	Messaging	30, 31, 32	Notification-system mock
8	Microservices and reliability	34, 39, 40	Mid-point mock with a peer
9	Social systems	53	News feed, chat, Twitter mocks
10	Media and files	54	YouTube, Dropbox mocks
11	Location and commerce	55	Uber, ticketing mocks
12	Data platforms	56, 16	Search, logging mocks
13	Senior depth	44, 47	Multi-region mock
14	Company preparation	90	Company-specific mocks
15	Practice library sweep	91	10 timed questions
16	Final review	Summaries of all above	3 full mocks; rubric self-score ≥ “Hire”

E.6 Track 3 — 24-week ML/AI infrastructure track (software engineer → ML/AI systems)

Budget: 10–12 hours/week. Assumes Level 3 on Track A.

Table E.4 — ML/AI track.

Wk	Chapters	Deliverable
1–2	4.9, 57	Metrics notebook; A/B test sample-size calculator
3	58	PyTorch MLP; parameter/activation memory worksheet
4	59, 60	Architecture diagram of an end-to-end ML system
5–6	61, 29	Lakehouse table + streaming feature job
7	62, 63	Feature store with PIT-correct training set
8–9	64	GPU profiling exercise; roofline worksheet
10–11	65	Multi-GPU training run; parallelism plan for a 70B model
12	66	Model server with dynamic batching and autoscaling
13	67	Quantization + ONNX/TensorRT comparison
14	68	Two-tower retrieval + ranker
15	69, 70	CI/CD/CT pipeline with drift-triggered retraining
16	71	Fairness audit and model card
17	72	Transformer from scratch (small)
18–19	73	LLM serving benchmark; capacity plan
20	74, 75	Production RAG with evaluation harness
21	76, 77	LoRA fine-tune; tool-using agent
22	78, 79	Eval pipeline; model gateway with routing and fallback
23	80, 81–85	Two case studies written as design docs
24	86, 92	Four ML/AI mock interviews

E.7 Monthly milestone review template

1. **Recall:** answer every knowledge-checkpoint question for the month's chapters without notes; mark misses for spaced repetition.

2. **Redraw:** reproduce the month’s three most important architecture diagrams from memory; compare with the book.
3. **Quantify:** solve three estimation problems from Ch 89 in under five minutes each.
4. **Design:** solve one design challenge aloud in 45 minutes; self-score with the Ch 88 rubric.
5. **Build:** demo the month’s project to someone else, including one injected failure.
6. **Reflect:** write half a page in the design journal — what is still unclear, which open question to research next.

2. Concept Dependency Map

This map shows which chapters must be understood before others. It drives the reading order, the “Prerequisites” section of every chapter, and the weekly roadmap (Appendix E). An arrow `A --> B` means “A is a prerequisite of B”.

2.1 Part-level dependency graph

Figure P.1 — Part-level dependencies.

WARNING / TRADE-OFF

Diagram failed to render. Source:

```
flowchart TD
  P1["Part I<br/>Design Thinking + Maths<br/>Ch 1-4"] --> P2["Part II<br/>CS
Foundations<br/>Ch 5-10"]
  P2 --> P3["Part III<br/>Data Systems<br/>Ch 11-17"]
  P2 --> P4["Part IV<br/>Scalability + Performance<br/>Ch 18-23"]
  P3 --> P4
  P3 --> P5["Part V<br/>Distributed Systems<br/>Ch 24-29"]
  P4 --> P5
  P5 --> P6["Part VI<br/>Messaging + Events<br/>Ch 30-32"]
  P2 --> P7["Part VII<br/>Software Architecture<br/>Ch 33-38"]
  P6 --> P7
  P4 --> P8["Part VIII<br/>Production Infra + Ops<br/>Ch 39-43"]
  P7 --> P8
  P5 --> P9["Part IX<br/>Advanced + Expert<br/>Ch 44-51"]
  P8 --> P9
  P8 --> P10["Part X<br/>Traditional Case Studies<br/>Ch 52-56"]
  P6 --> P10
  P1 --> P11["Part XI<br/>ML Foundations<br/>Ch 57-60"]
  P11 --> P12["Part XII<br/>ML Data Infra<br/>Ch 61-63"]
  P5 --> P12
  P11 --> P13["Part XIII<br/>Training Infra<br/>Ch 64-65"]
  P11 --> P14["Part XIV<br/>Serving + Optimization<br/>Ch 66-68"]
  P13 --> P14
  P12 --> P15["Part XV<br/>MLOps + Responsible ML<br/>Ch 69-71"]
  P14 --> P15
  P14 --> P16["Part XVI<br/>LLM + AI Systems<br/>Ch 72-80"]
  P13 --> P16
  P15 --> P17["Part XVII<br/>ML/AI Case Studies<br/>Ch 81-86"]
  P16 --> P17
  P10 --> P18["Part XVIII<br/>Interview Prep<br/>Ch 87-92"]
  P17 --> P18
  P10 --> P19["Part XIX<br/>Projects<br/>Ch 93"]
  P17 --> P19
```

Reading the diagram. Each box is a part of the book; arrows point from prerequisite to dependent. Two independent entry points exist after Part I: the traditional track (Parts II→X) and the ML track (Part XI onward). The ML track rejoins the traditional track at Part XII (data infrastructure needs distributed systems from Part V) and at Part XVI (LLM serving needs serving infrastructure from Parts IV and VIII via Part XIV). The interview and project parts sit at the end because they integrate everything.

2.2 Concept-level dependency graph (critical path)

Figure P.2 — Critical concept path from first principles to LLM infrastructure.

WARNING / TRADE-OFF

Diagram failed to render. Source:

```
flowchart LR
  M["Ch 4 Little's Law,<br/>percentiles, queueing"] --> PERF["Ch 23
Performance<br/>engineering"]
  CA["Ch 5 Memory hierarchy"] --> SE["Ch 13 Storage engines"]
  OS["Ch 6 Threads, I/O models"] --> RT["Ch 9 Real-time<br/>connections"]
  NET["Ch 7 TCP, HTTP, TLS"] --> WEB["Ch 8 APIs, auth"]
  NET --> LB["Ch 19 Load balancing"]
  RDB["Ch 11 Indexes,<br/>transactions"] --> SE
  SE --> SCALE["Ch 14 Replication,<br/>sharding"]
  SCALE --> CH["Ch 17 Consistent<br/>hashing"]
  CH --> CACHE["Ch 20 Distributed<br/>caching"]
  SCALE --> CONS["Ch 25 Consistency<br/>models"]
  CLK["Ch 24 Clocks,<br/>failures"] --> CONS
  CONS --> RAFT["Ch 26 Raft, quorums"]
  RAFT --> KAFKA["Ch 31 Kafka ISR"]
  RAFT --> TX["Ch 28 Distributed<br/>transactions"]
  KAFKA --> EDA["Ch 32 Outbox,<br/>sagas, CQRS"]
  TX --> EDA
  LA["Ch 4 Linear algebra"] --> MLF["Ch 57-58 ML + DL<br/>fundamentals"]
  CA --> GPU["Ch 64 GPU<br/>architecture"]
  MLF --> GPU
  GPU --> DT["Ch 65 Distributed<br/>training"]
  MLF --> LLM["Ch 72 Transformers,<br/>KV cache"]
  GPU --> LLMI["Ch 73 LLM inference"]
  LLM --> LLMI
  PERF --> LLMI
  CACHE --> LLMI
  MLF --> VEC["Ch 74 Vector search"]
  CH --> VEC
  VEC --> RAG["Ch 75 RAG"]
  LLMI --> RAG
  RAG --> AG["Ch 77 Agents"]
  EDA --> AG
```

Reading the diagram. This is the longest chain of concepts the book builds. For example, to understand continuous batching and KV-cache paging in LLM inference (Ch 73) you need GPU memory hierarchy (Ch 64), the attention/KV-cache mechanism (Ch 72), queueing and tail-latency reasoning (Ch 4, Ch 23) and caching concepts (Ch 20). Agentic systems (Ch 77) depend on both RAG and on durable workflow patterns from event-driven architecture (Ch 32), because production agents are long-running, retryable workflows.

2.3 Chapter prerequisite table

Table P.1 — Hard prerequisites per chapter (soft prerequisites are named inside each chapter).

Ch	Title	Hard prerequisites
1	What Is System Design?	—
2	How to Think Like a System Designer	1
3	Learning Method	1
4	Quantitative Foundations	— (high-school algebra)
5	Computer Architecture	4
6	Operating Systems	5
7	Networking Fundamentals	4, 6
8	Web Application Fundamentals	7
9	Real-Time Communication	6, 7, 8
10	Security Fundamentals	7, 8
11	Relational Databases	4, 5
12	NoSQL Databases	11
13	Storage Engine Internals	5, 6, 11
14	Database Scaling	11, 12, 13
15	Object/Block/File Storage	5, 6, 14
16	Search Systems	11, 13, 14
17	Data Structures for Scale	4, 14
18	Scalability	2, 4, 8
19	Load Balancing	7, 18
20	Caching	5, 14, 18
21	CDNs	7, 20
22	API and Traffic Management	4, 8, 18
23	Performance Engineering	4, 6, 18
24	Distributed Systems Fundamentals	7, 14
25	Consistency and Availability	14, 24
26	Consensus and Coordination	24, 25
27	Replication and Fault Tolerance	14, 25, 26
28	Distributed Transactions	11, 26, 27
29	Distributed Data Processing	14, 24
30	Messaging Fundamentals	8, 24
31	Kafka Deep Dive	13, 26, 30
32	Event-Driven Patterns	28, 30, 31

Ch	Title	Hard prerequisites
33	Architecture Patterns	8, 30
34	Microservices	22, 32, 33
35	Object-Oriented Design	— (basic programming)
36	SOLID	35
37	Design Patterns	35, 36
38	UML and LLD	35–37
39	Reliability and Resilience	4, 22, 27
40	Observability	4, 34, 39
41	Cloud Computing	7, 15
42	Containers and Kubernetes	6, 41
43	Deployment and Infrastructure	42
44	Multi-Region Systems	25, 27, 39
45	Advanced Consistency and BFT	25, 26
46	Distributed Scheduling	26, 42
47	Multi-Tenancy	22, 42
48	Edge and Serverless	21, 41
49	Large-Scale Data Infra and HPC	15, 29
50	Security Architecture and Privacy	10, 34
51	Cost and Sustainability	41
52–56	Traditional case studies	Parts I–VIII (each case lists specifics)
57	ML Fundamentals	4
58	Deep Learning Essentials	57
59	ML Lifecycle	57
60	ML System Architecture	18, 30, 59
61	Data Engineering for ML	29, 60
62	Feature Stores	20, 61
63	Data Labeling and Quality	59, 61
64	GPU Architecture	5, 58
65	Distributed Training	7, 58, 64
66	Model Serving	19, 42, 60
67	Model Optimization	64, 66
68	Recommendation Architecture	62, 66, 74 (ANN preview)

Ch	Title	Hard prerequisites
69	MLOps	43, 59, 66
70	ML Monitoring	40, 57, 69
71	Responsible ML	57, 69
72	LLM Fundamentals	4.9, 58
73	LLM Inference	23, 64, 66, 72
74	Vector Search	4.9, 16, 72
75	RAG	16, 73, 74
76	LLM Adaptation	65, 72
77	Agents	32, 75
78	LLM Evaluation	57, 75, 77
79	Production AI Infrastructure	47, 73, 78
80	Large-Scale AI Infrastructure	46, 65, 79
81–86	ML/AI case studies	Parts XI–XVI (each case lists specifics)
87–92	Interview preparation	Part X and/or Part XVII
93	Projects	Per project (listed in project)

2.4 Reading paths

Goal	Path
Absolute beginner, traditional track	1 → 3 → 4 → 5 → 6 → 7 → 8 → 11 → 18 → 19 → 20 → 30 → 33 → 52.1
Interview in 8 weeks (experienced backend engineer)	2, 4 (skim), 14, 17, 18–22, 24–27, 30–32, 34, 39, 52–56, 87–89, 91
ML engineer moving into ML system design	4.9–4.10, 57–60, 61–62, 64–66, 68–70, 81–85, 88, 92
Backend engineer moving into AI infrastructure	4, 23, 57–58, 64, 66, 72–75, 77–79, 86
Staff/principal preparation	28, 39, 44–51, 80, 88 (rubrics), 90